

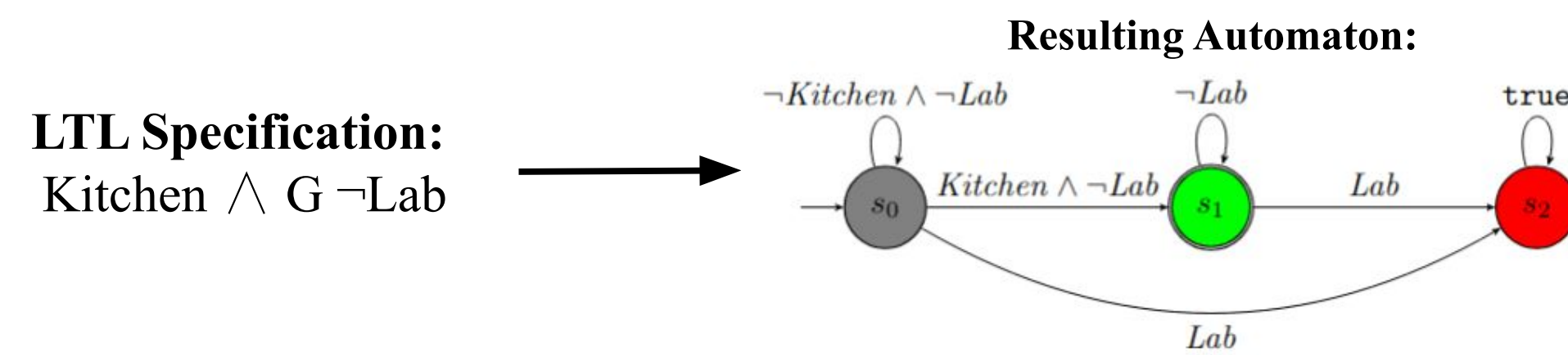
## Abstract

As Large Language Models (LLMs) continue to advance in ability and scope, their role in the next generation of robotic systems is rapidly expanding. Such systems are increasingly being deployed in real-world environments with tangible consequences, and the ability to generate safe and reliable plans that consider and account for environmental uncertainties is critical to ensuring the well being of both humans and robots. LLMs have shown promising capabilities in generating safe and feasible plans for robots, and additional research has started to emerge investigating the use of LLMs to verify the safety of generated plans.

Formal methods such as Linear Temporal Logic (LTL) have successfully been used to ensure generated plans meet mission constraints; however, they do not tend to generalize well to unseen or unpredicted scenarios due to their rigid and exact definitions. LLMs have shown an impressive ability to provide broader contextual understanding to a large spectrum of tasks, but cannot be fully relied upon to provide safe and feasible solutions due to their probabilistic nature. In this MQP, we aim to combine the strengths of classical and data-driven methods of planning by using the generalized problem solving abilities of LLMs to inform traditional LTL based planning, creating workable and safe plans that account for the wider context of a scenario while still fulfilling mission specifications and following predefined rules.

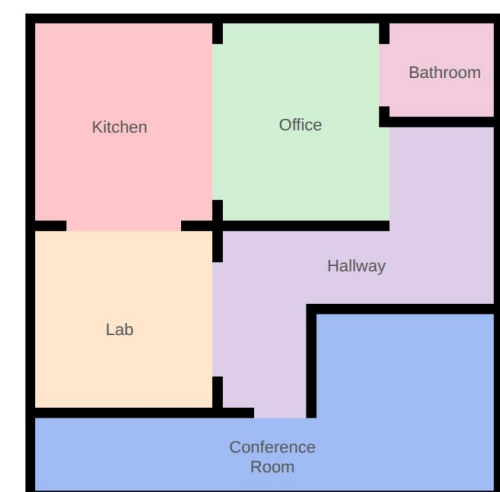
## Formal Methods for Task Planning

Representing a Linear Temporal Logic (LTL) task as a Büchi automaton

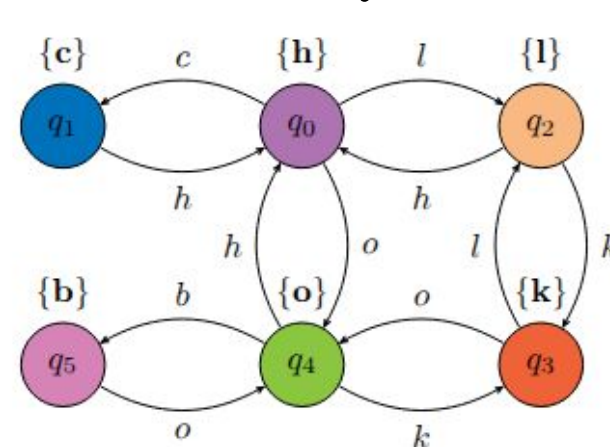


Encoding the environment as a Transition System (TS)

**Labelled Environment**

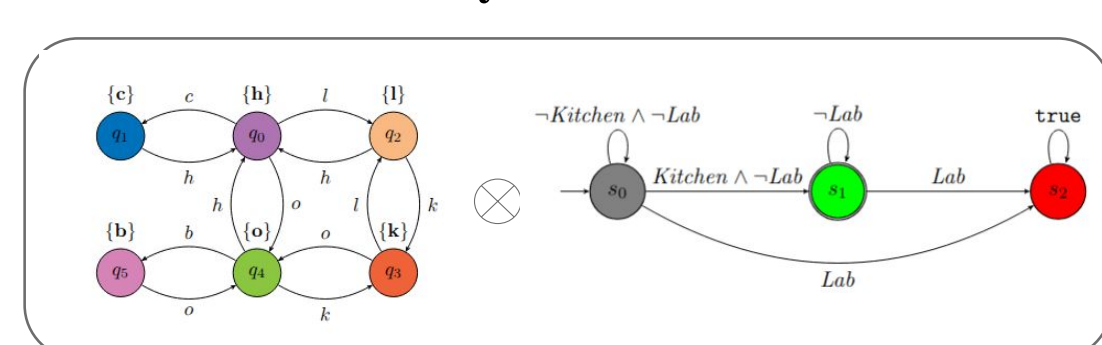


**Transition System**

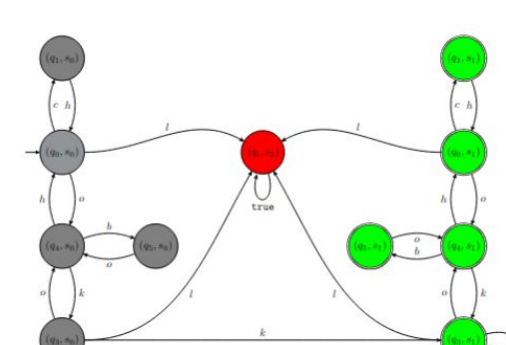


Calculating the **product automaton** between the TS and the Büchi automaton

**Transition System & Automaton**

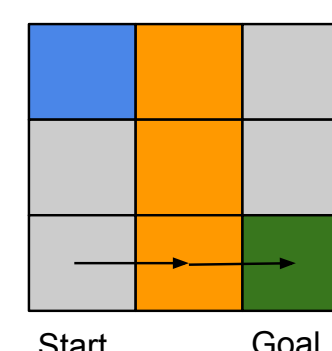


**Product Automaton**

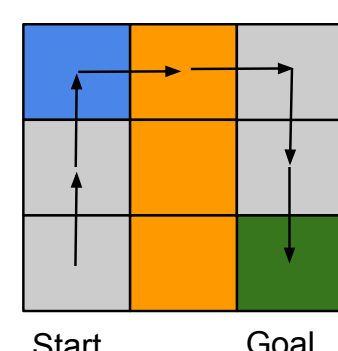


Using the product automaton to inform **state aware A\***

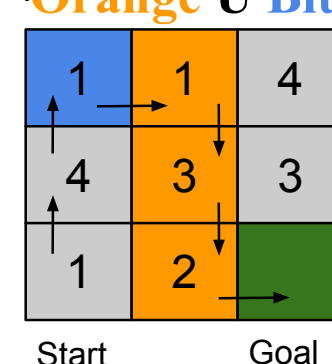
**Path traced by regular A\***



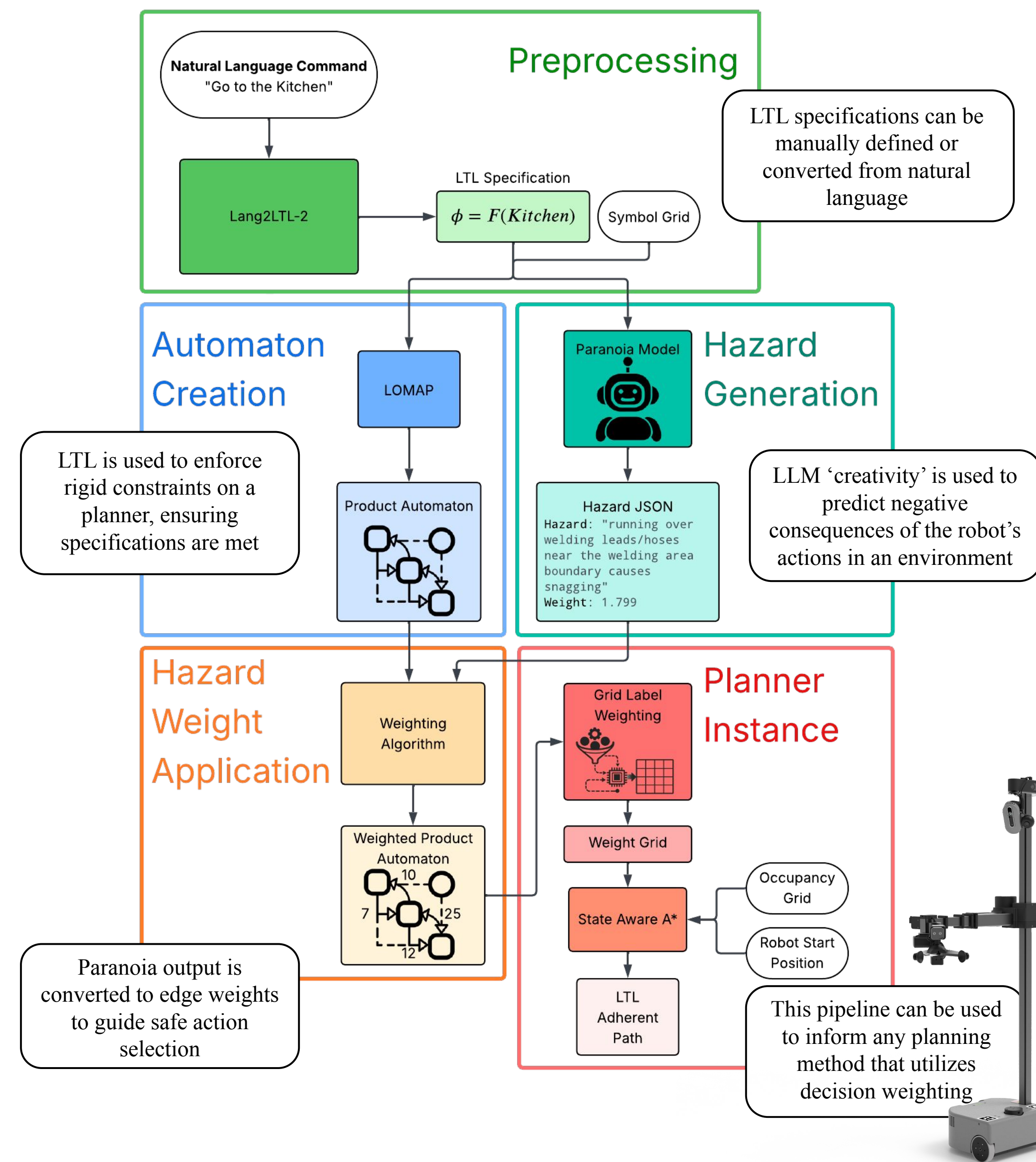
**Path produced by SAA\* with spec ¬Orange U Blue**



**Path produced by SAA\* (weighted) with spec ¬Orange U Blue**



## Think Again Pipeline



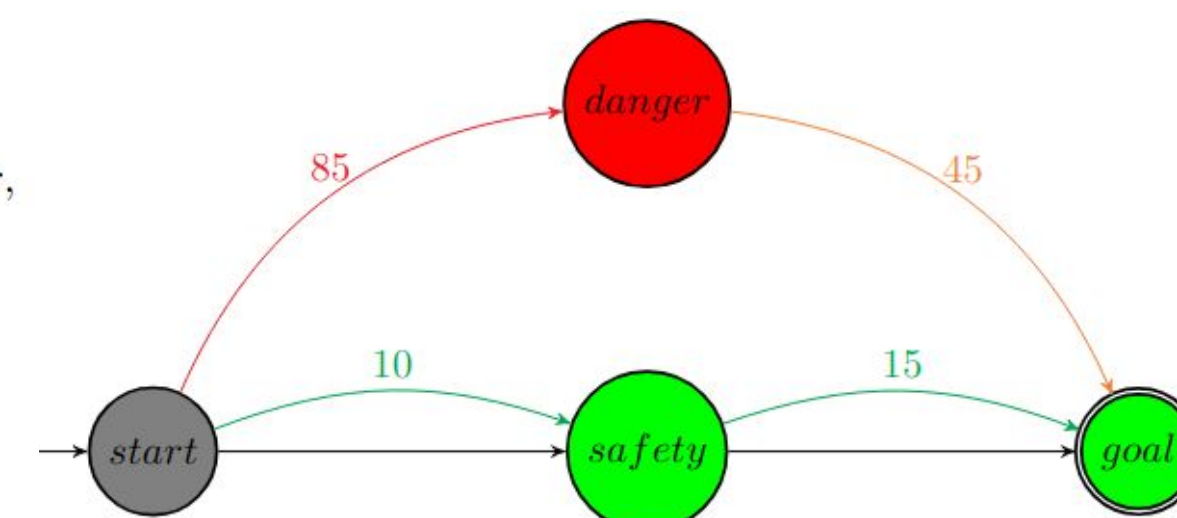
## Weighting Algorithm

$$\text{score} = p_i \cdot (\alpha \cdot v(h_{\text{minor}}) + \beta \cdot v(h_{\text{major}}) + \gamma \cdot v(h_{\text{death}}) + \alpha \cdot v(r_{\text{minor}}) + \beta \cdot v(r_{\text{major}}) + \gamma \cdot v(r_{\text{death}}))$$

where:

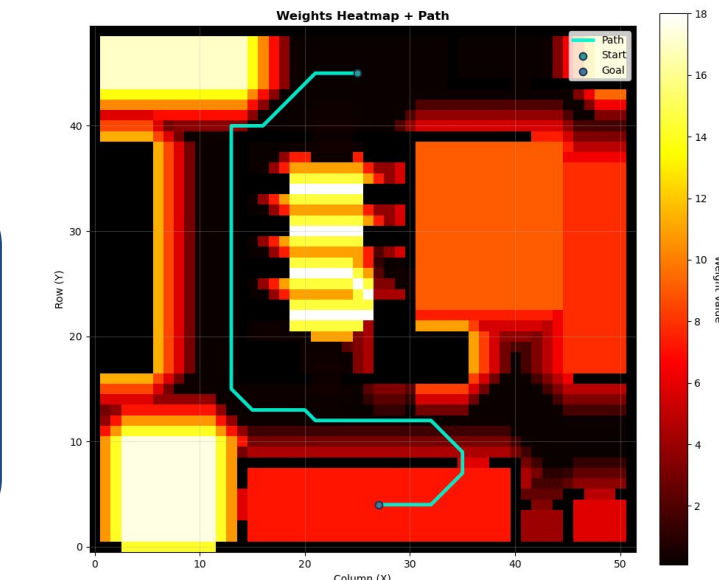
- $p_i$  is the probability that risk  $i$  occurs
- $\alpha, \beta, \gamma$  are the weight values for the minor, major and death categories
- $v(h_x)$  is the value assigned by Paranoia for each category for risk  $i$

**Example Weighted Product Automaton**

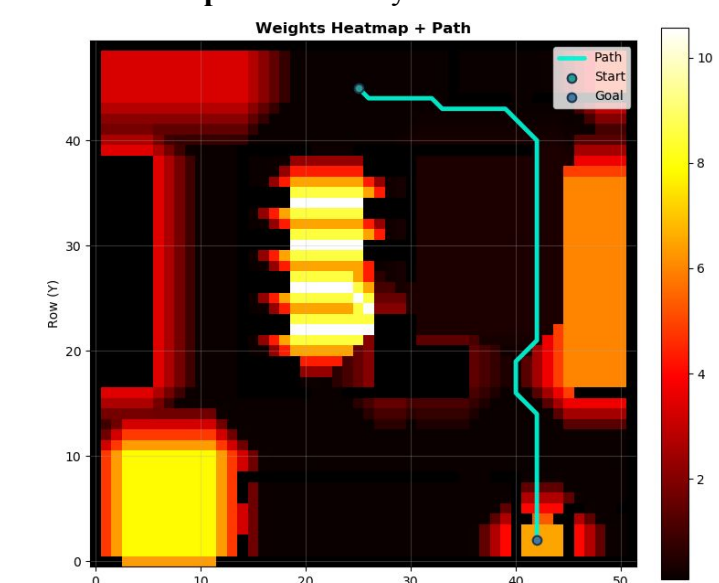


## Results

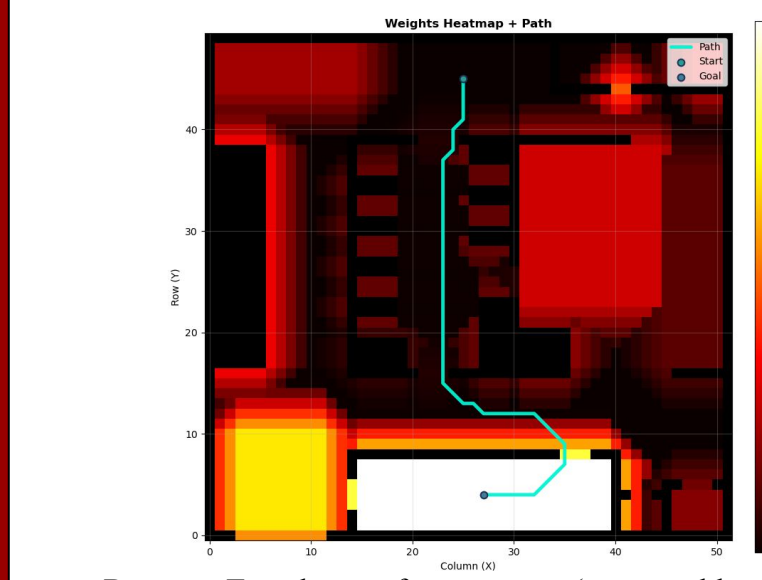
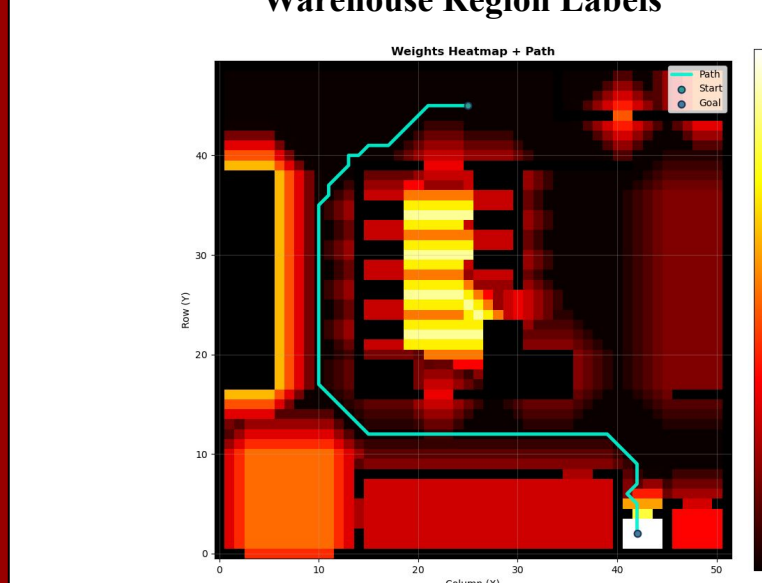
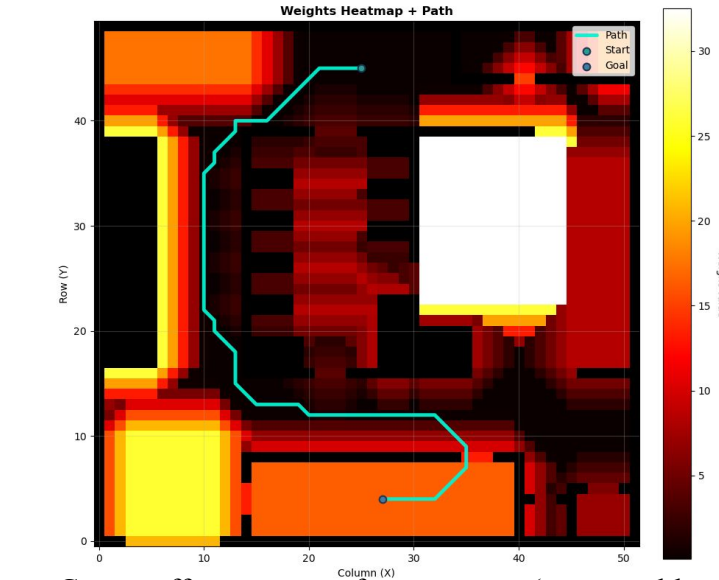
With no direct prompting, Think Again successfully avoids regions of higher risk (welding area)



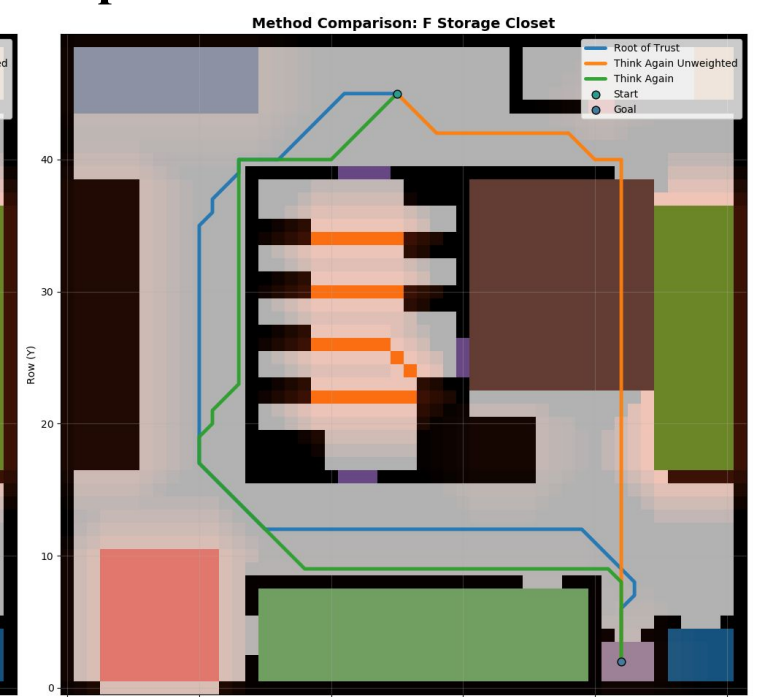
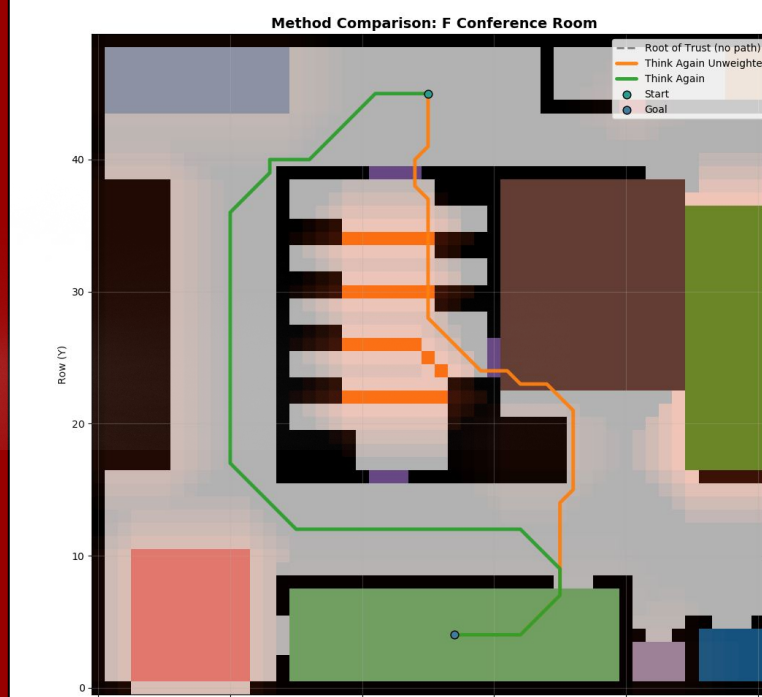
Even when certain regions are certified safe, Think Again avoids them when specified in the LTL task



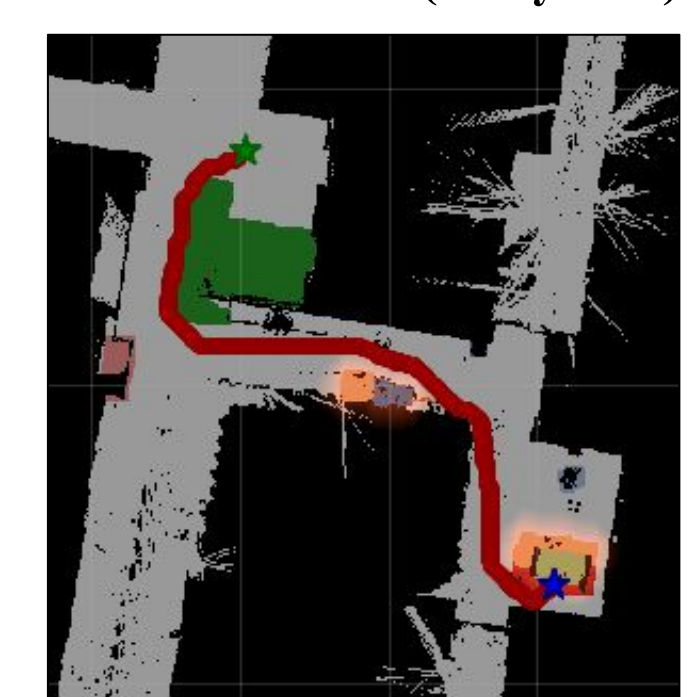
Think Again can account for nuances of specific tasks and adjust plans accordingly without direct prompting



## Method Comparisons



## Hardware Demo (Unity Hall)



## Conclusion

**Think Again :**

- Allows rigid, formal constraints and environmental context to be considered in the planning process
- Enables the planner to account for dangers it has not explicitly been informed about beforehand
- Offers better plan safety and feasibility than comparison methods in example scenarios
- Can be extended to a range of other planning problems of various scopes